

BAŞLIK: PEKİŞTİRMELİ ÖĞRENME (REINFORCEMENT LEARNING): TEMEL KAVRAMLAR VE MATEMATİKSEL ÇERÇEVE

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: PEKİŞTİRMELİ ÖĞRENME NEDİR?

Tanım: Bir ajanın (agent), bir ortam (environment) içinde deneme-yanılma yoluyla, aldığı ödülleri (reward) uzun vadede maksimize edecek şekilde davranmayı öğrendiği makine öğrenmesi paradigmasıdır.

Pekiştirmeli Öğrenme (Reinforcement Learning - RL), davranışçılık psikolojisinden (behaviorism) doğrudan ilham alır. Bir köpeği eğitirken ona hangi kasın hangi sinyali tetiklediğini anlatmayız; doğru davranışı yaptığında ödül (mama) veririz, yanlış davranışta ödül vermeyiz. Zamanla köpek, hangi davranışın ödülle sonuçlandığını kendi deneyimiyle öğrenir. Pekiştirmeli öğrenmedeki ajan da tam olarak bunu yapar: doğru cevabı gösteren bir öğretmen (supervised learning'deki etiketler gibi) yoktur; sadece "bu iyiydi" ya da "bu kötüydü" anlamına gelen sayısal bir geri bildirim (ödül sinyali) vardır.

1.1. Üç Öğrenme Türü Arasındaki Fark

Gözetimli öğrenmede (supervised learning) modele hem soru (X) hem doğru cevap (y) verilir; model bu ikisi arasındaki ilişkiyi taklit etmeye çalışır. Gözetimsiz öğrenmede (unsupervised learning) sadece veri vardır, doğru cevap yoktur; model verinin içindeki gizli yapıyı bulur. Pekiştirmeli öğrenmede ise ne hazır bir doğru cevap etiketi, ne de statik bir veri seti vardır — ajan, kendi eylemleriyle ortamı etkileyerek kendi veri setini (deneyimini) zaman içinde kendisi üretir ve gecikmeli, seyrek bir ödül sinyalinden öğrenir. Bu "gecikmeli ödül" (delayed reward) özelliği pekiştirmeli öğrenmeyi diğer iki türden ayıran en kritik noktadır: satranç oynayan bir ajan, oyunun 10. hamlesinde yaptığı bir hatanın bedelini genelde ancak 40. hamlede, oyunu kaybederek öder.

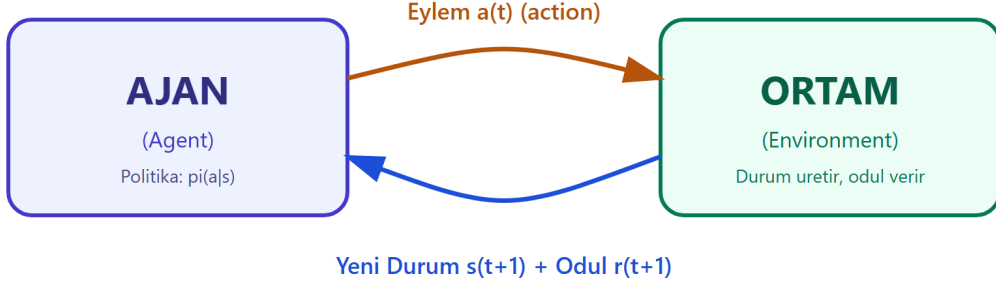
1.2. Temel Felsefe: Ödül Maksimizasyonu (Reward Maximization)

Pekiştirmeli öğrenmenin nihai amacı, tek bir adımda en yüksek ödülü almak değil, bir epizot (episode) boyunca — hatta sonsuz ufuklu problemlerde tüm gelecek boyunca — toplanan ödüllerin toplamını (genellikle "getiri" / return olarak adlandırılır) maksimize etmektir. Bu yüzden bazen kısa vadede ödül kaybettiren bir eylem (örn. satrançta bir taş feda etmek), uzun vadede daha büyük bir ödüle (oyunu kazanmak) götürüyorsa "doğru" eylem olarak öğrenilir. Bu, RL'i klasik gözetimli öğrenmeden ayıran ikinci kritik fark: hedef fonksiyonu anlık değil, kümülatiftir.

BÖLÜM 2: TEMEL BİLEŞENLER (AJAN - ORTAM ETKİLEŞİMİ)

Her pekiştirmeli öğrenme probleminin çekirdeğinde, aşağıdaki şemada özetlenen döngüsel bir etkileşim vardır:

Ajan - Ortam Etkileşim Dongusu



Bu dongu, bolum sonuna (terminal state) kadar tekrar eder ve bir "epizot" (episode) olusturur.

Şekil 1 — Ajan, ortama bir eylem uygular; ortam yeni bir durum ve bir ödül döndürür. Bu döngü, bölüm sonuna kadar tekrar eder.

1. Ajan (Agent): Karar veren, öğrenen taraftır. Bir oyun oynayan yapay zeka, bir robot kolu ya da bir otonom aracın sürüş yazılımı ajan olabilir.
2. Ortam (Environment): Ajanın içinde bulunduğu, eylemlerinin sonuçlarını üreten dünyadır (Örn. bir satranç tahtası, bir video oyunu, gerçek bir fabrika hattı).
3. Durum (State - s): Ortamın belirli bir andaki tam tanımıdır. Ajan kararını, elindeki durum bilgisine bakarak verir (Örn. FrozenLake ortamında ajanın 4x4'lük ızgara üzerindeki konumu; CartPole ortamında arabanın konumu, hızı, direğin açısı ve açısal hızı).
4. Eylem (Action - a): Ajanın belirli bir durumda seçebileceği hareketlerden biridir (Örn. "sola git", "sağa git", "zıpla"). Eylem uzayı ayrık (discrete — sınırlı sayıda seçenek) ya da sürekli (continuous — bir gaz pedalının basınç derecesi gibi sonsuz olası değer) olabilir.
5. Ödül (Reward - r): Ortamın, ajanın az önce yaptığı eylemin ne kadar "iyi" olduğunu bildiren sayısal bir geri bildirimdir. Ödül fonksiyonunun tasarımı (reward shaping), pekiştirmeli öğrenmede başarıyı en çok etkileyen mühendislik kararlarından biridir — kötü tasarlanmış bir ödül, ajanı hiç istenmeyen bir davranışa "reward hacking" yoluyla yönlendirebilir.
6. Politika (Policy - π): Ajanın "beynidir". Bir durumu girdi olarak alıp hangi eylemi (veya eylemler üzerindeki olasılık dağılımını) seçeceğini belirleyen fonksiyondur: $\pi(a|s)$. Pekiştirmeli öğrenmenin nihai çıktısı, elde edilen bu π fonksiyonudur — "öğrenilen model" dediğimiz şey aslında bu politikadır.

BÖLÜM 3: MARKOV KARAR SÜREÇLERİ (MARKOV DECISION PROCESS - MDP)

Pekiştirmeli öğrenme problemleri matematiksel olarak bir Markov Karar Süreci (MDP) ile formüle edilir. Bir MDP, beşli bir demet (tuple) ile tanımlanır: (S, A, P, R, γ)

(S, A, P, R, γ)

- S : Olası tüm durumların kümesi.
- A : Olası tüm eylemlerin kümesi.
- P : Geçiş olasılığı fonksiyonu $P(s'|s,a)$ — s durumunda a eylemi yapıldığında s' durumuna geçme olasılığı.

- R: Ödül fonksiyonu $R(s,a,s')$ — bu geçiştten alınan ödül.
- γ (gamma, indirim faktörü): 0 ile 1 arasında bir sayıdır ve gelecekteki ödüllerin bugünkü değerini belirler.

3.1. Markov Özelliği (Markov Property)

"Markov" ismi, bir sistemin geleceğinin, yalnızca şu anki durumuna bağlı olduğu, geçmişteki tüm durum/eylem geçmişinden bağımsız olduğu varsayımından gelir: $P(s(t+1) | s(t), a(t)) = P(s(t+1) | s(t), a(t), s(t-1), a(t-1), \dots, s(0))$. Basitçe: "şu anki durum, geleceği tahmin etmek için gereken tüm bilgiyi zaten içerir; geçmiş hatırlamaya gerek yoktur." Örneğin satrançta, tahtanın şu anki hali, bir sonraki en iyi hamleyi belirlemek için yeterlidir; taşların oraya hangi sırayla geldiği önemli değildir.

3.2. İndirim Faktörü (Discount Factor - γ) Neden Var?

İnsanlar için bile bugün eline geçen 100 TL, bir yıl sonra alınacak 100 TL'den daha değerlidir. Aynı mantık RL'de de uygulanır: γ katsayısı 1'e ne kadar yakınsa ajan o kadar "sabırlıdır" ve uzak gelecekteki ödülleri neredeyse bugünküyle eşit önemser; 0'a ne kadar yakınsa o kadar "açgözlüdür" (short-sighted) ve sadece anlık ödülü önemser. Toplam getiri (return) şu formülle hesaplanır:

$$G(t) = r(t+1) + \gamma \cdot r(t+2) + \gamma^2 \cdot r(t+3) + \dots = \sum \gamma^k \cdot r(t+k+1)$$

γ ayrıca sonsuz ufuklu (never-ending) problemlerde toplamın matematiksel olarak sonlu bir sayıya yakınsamasını (converge) da garanti eder — γ olmasaydı, sonsuz sayıda pozitif ödülün toplamı sonsuza giderdi ve karşılaştırma yapılamazdı.

BÖLÜM 4: KEŞİF VS. SÖMÜRÜ (EXPLORATION VS. EXPLOITATION)

Pekiştirmeli öğrenmenin belki de en özgün ikilemi budur ve ne gözetimli ne de gözetimsiz öğrenmede bir karşılığı vardır. Şu senaryoyu düşünün: Sürekli gittiğiniz bir mahallede, her gün aynı lokantaya gidip yediğiniz yemekten memnunsunuz (bildiğiniz güvenli seçenek — 7/10 puan). Ama iki sokak ötede hiç denemediğiniz yeni bir lokanta var; belki 9/10 puan verecek kadar lezzetli, belki de berbat çıkacak.

- Sömürü (Exploitation): Şu ana kadar en iyi olduğunu bildiğiniz seçeneği (eski lokanta) tekrar seçmek. Garanti ama potansiyel olarak optimalin altında bir ödül.
- Keşif (Exploration): Bilmediğiniz bir seçeneği (yeni lokanta) denemek. Riskli ama daha iyi bir strateji keşfetme potansiyeli taşır.

Bir ajan sadece sömürü yaparsa, tesadüfen erken keşfettiği vasat bir stratejide sıkışıp kalır ve gerçek optimum stratejiyi asla bulamaz. Sadece keşif yaparsa, öğrendiği hiçbir bilgiyi kullanmadığı için rastgele davranmaktan farksız kalır ve performansı asla iyileşmez. Bu yüzden iyi bir RL algoritması ikisi arasında bir denge kuralıdır.

4.1. Epsilon-Açgözlü (ϵ -greedy) Strateji

En yaygın ve en basit çözüm epsilon-açgözlü (ϵ -greedy) yaklaşımdır: Ajan her adımda ϵ olasılıkla tamamen rastgele bir eylem seçer (keşif), $(1-\epsilon)$ olasılıkla da o ana kadar öğrendiği en iyi eylemi seçer (sömürü). Eğitimin başında ϵ genellikle yüksek tutulur (örn. 1.0 — tamamen rastgele davran, dünyayı tanı) ve eğitim ilerledikçe kademeli olarak küçültülür (epsilon decay) — ajan dünyayı tanıdıkça öğrendiklerine daha çok güvenmeye başlar.

SONUÇ: UYGULAMA ALANLARI

Bu bölümde öğrendiğimiz Ajan-Ortam-Durum-Eylem-Ödül-Politika çerçevesi ve MDP formalizmi, bugün şu alanlarda hayat buluyor:

1. Oyun Oynayan Yapay Zekalar: AlphaGo/AlphaZero (Go, Satranç), Atari oyunlarını insan üstü seviyede oynayan DQN ajanları, OpenAI Five (Dota 2).
2. Otonom Sistemler: Kendi kendine giden araçların şerit takibi ve manevra kararları, insansız hava araçlarının rota planlaması.
3. Robotik: Bir robot kolunun nesneleri kavrama hareketini deneme-yanılma ile öğrenmesi.
4. Kaynak Yönetimi: Veri merkezlerinde soğutma sistemlerinin enerji tüketimini optimize etmek (Google DeepMind'ın veri merkezi projesi), elektrik şebekesi yük dengeleme.
5. Büyük Dil Modelleri (LLM'lerin İnce Ayarı): RLHF (Reinforcement Learning from Human Feedback) — ChatGPT benzeri modellerin insan geri bildirimlerinden ödül sinyali üreterek ince ayar yapılması (bkz. 20-Generative_AI_and_Prompt_Engineer modülü).
6. Finans: Portföy optimizasyonu ve algoritmik ticaret stratejilerinin risk-ödül dengesini öğrenmesi.

Bir sonraki dosyada (21.2.1), bu teorik çerçeveyi somut bir algoritmaya — Q-Learning'e — dökeceğiz ve FrozenLake ortamı üzerinde uygulayacağız.